Program 4

Develop a menu-driven program to implement the following operations on array storage representation with static and dynamic memory allocation:
i) Insert by position
ii) Delete by position
iii) Insert by key
iv) Delete by key
v) Insert by order
vi) Search by key
vii) Search by position
viii) Reverse the contents.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

/* Function declarations */
void display(int a[], int n);
void insertPosition(int a[], int *n, int pos, int value);
void deletePosition(int a[], int *n, int pos);
void insertKey(int a[], int *n, int key, int value);
void deleteKey(int a[], int *n, int key);
void insertOrder(int a[], int *n, int value);
void searchKey(int a[], int n, int key);
void searchPosition(int a[], int n, int pos);
void reverseArray(int a[], int n);

int main()
{
   int choice, operation;
   int n, i;
   int pos, key, value;

   int staticArray[MAX];
   int *a;

   /* Choose memory allocation */
   printf("Choose Memory Allocation:\n");
   printf("1. Static Memory Allocation\n");
   printf("2. Dynamic Memory Allocation\n");
   printf("Enter your choice: ");
   scanf("%d", &choice);
```

```
/* Static allocation */
if (choice == 1)
{
   a = staticArray;

   printf("\nEnter number of elements: ");
   scanf("%d", &n);

   printf("Enter %d elements:\n", n);
   for (i = 0; i < n; i++)
      scanf("%d", &a[i]);
}

/* Dynamic allocation */
else if (choice == 2)
{
   printf("\nEnter number of elements: ");
   scanf("%d", &n);

   a = (int *)malloc(n * sizeof(int));

   if (a == NULL)
   {
      printf("Memory allocation failed!\n");
      return 1;
   }

   printf("Enter %d elements:\n", n);
   for (i = 0; i < n; i++)
      scanf("%d", &a[i]);
}
else
{
   printf("Invalid choice!\n");
   return 0;
}

/* Menu */
do
{
   printf("\n\n----- ARRAY OPERATIONS -----\n");
   printf("1. Insert by Position\n");
   printf("2. Delete by Position\n");
   printf("3. Insert by Key\n");
   printf("4. Delete by Key\n");
   printf("5. Insert by Order\n");
   printf("6. Search by Key\n");
```

```
printf("7. Search by Position\n");
printf("8. Reverse the Contents\n");
printf("9. Display\n");
printf("10. Exit\n");

printf("Enter your choice: ");
scanf("%d", &operation);

switch (operation)
{
   case 1:
      printf("Enter position: ");
      scanf("%d", &pos);

      printf("Enter value: ");
      scanf("%d", &value);

      if (choice == 1 && n == MAX)
         printf("Array is full!\n");
      else
      {
         if (choice == 2)
            a = (int *)realloc(a, (n + 1) * sizeof(int));

         insertPosition(a, &n, pos, value);
      }
      break;

   case 2:
      printf("Enter position to delete: ");
      scanf("%d", &pos);

      deletePosition(a, &n, pos);

      if (choice == 2)
         a = (int *)realloc(a, n * sizeof(int));

      break;

   case 3:
      printf("Enter key: ");
      scanf("%d", &key);

      printf("Enter value to insert: ");
      scanf("%d", &value);

      if (choice == 1 && n == MAX)
```

```
      printf("Array is full!\n");
    else
    {
      if (choice == 2)
        a = (int *)realloc(a, (n + 1) * sizeof(int));

      insertKey(a, &n, key, value);
    }
    break;

  case 4:
    printf("Enter key to delete: ");
    scanf("%d", &key);

    deleteKey(a, &n, key);

    if (choice == 2)
      a = (int *)realloc(a, n * sizeof(int));

    break;

  case 5:
    printf("Enter value: ");
    scanf("%d", &value);

    if (choice == 1 && n == MAX)
      printf("Array is full!\n");
    else
    {
      if (choice == 2)
        a = (int *)realloc(a, (n + 1) * sizeof(int));

      insertOrder(a, &n, value);
    }
    break;

  case 6:
    printf("Enter key to search: ");
    scanf("%d", &key);

    searchKey(a, n, key);
    break;

  case 7:
    printf("Enter position to search: ");
    scanf("%d", &pos);
```

```
            searchPosition(a, n, pos);
            break;

        case 8:
            reverseArray(a, n);
            printf("Array reversed successfully.\n");
            break;

        case 9:
            display(a, n);
            break;

        case 10:
            printf("Program terminated.\n");
            break;

        default:
            printf("Invalid choice!\n");
      }

   } while (operation != 10);

   /* Free dynamic memory */
   if (choice == 2)
      free(a);

   return 0;
}


/* Display array */
void display(int a[], int n)
{
   int i;

   printf("Array: ");

   for (i = 0; i < n; i++)
      printf("%d ", a[i]);

   printf("\n");
}


/* Insert by position */
void insertPosition(int a[], int *n, int pos, int value)
{
```

```
    int i;

    if (pos < 1 || pos > *n + 1)
    {
        printf("Invalid position!\n");
        return;
    }

    for (i = *n; i >= pos; i--)
        a[i] = a[i - 1];

    a[pos - 1] = value;

    (*n)++;

    printf("Element inserted successfully.\n");
}


/* Delete by position */
void deletePosition(int a[], int *n, int pos)
{
    int i;

    if (pos < 1 || pos > *n)
    {
        printf("Invalid position!\n");
        return;
    }

    for (i = pos - 1; i < *n - 1; i++)
        a[i] = a[i + 1];

    (*n)--;

    printf("Element deleted successfully.\n");
}


/* Insert by key */
void insertKey(int a[], int *n, int key, int value)
{
    int i, position = -1;

    /* Find key */
    for (i = 0; i < *n; i++)
    {
```

```
    if (a[i] == key)
    {
      position = i;
      break;
    }
  }

  if (position == -1)
  {
    printf("Key not found!\n");
    return;
  }

  /* Shift elements */
  for (i = *n; i > position + 1; i--)
    a[i] = a[i - 1];

  a[position + 1] = value;

  (*n)++;

  printf("Element inserted after key %d.\n", key);
}


/* Delete by key */
void deleteKey(int a[], int *n, int key)
{
  int i, position = -1;

  /* Find key */
  for (i = 0; i < *n; i++)
  {
    if (a[i] == key)
    {
      position = i;
      break;
    }
  }

  if (position == -1)
  {
    printf("Key not found!\n");
    return;
  }

  /* Shift elements */
```

```
    for (i = position; i < *n - 1; i++)
        a[i] = a[i + 1];

    (*n)--;

    printf("Key %d deleted successfully.\n", key);
}


/* Insert by order */
void insertOrder(int a[], int *n, int value)
{
    int i;

    /* Find correct position */
    i = *n - 1;

    while (i >= 0 && a[i] > value)
    {
        a[i + 1] = a[i];
        i--;
    }

    a[i + 1] = value;

    (*n)++;

    printf("Element inserted in ascending order.\n");
}


/* Search by key */
void searchKey(int a[], int n, int key)
{
    int i;

    for (i = 0; i < n; i++)
    {
        if (a[i] == key)
        {
            printf("Key %d found at position %d.\n", key, i + 1);
            return;
        }
    }

    printf("Key not found!\n");
}
```

```
/* Search by position */
void searchPosition(int a[], int n, int pos)
{
   if (pos < 1 || pos > n)
   {
      printf("Invalid position!\n");
      return;
   }

   printf("Element at position %d = %d\n", pos, a[pos - 1]);
}


/* Reverse array */
void reverseArray(int a[], int n)
{
   int i, temp;

   for (i = 0; i < n / 2; i++)
   {
      temp = a[i];
      a[i] = a[n - 1 - i];
      a[n - 1 - i] = temp;
   }
}
```